

Scheduling Chip Operations with Tabu Search

Using fewer hardware units when designing a chip

Soaib Ferdous

Student ID: 1232368 | Group B, Team 6

HW/SW Co-Design Seminar — Hochschule Hamm-Lippstadt

Why This Matters

Building a chip means building real hardware for every operation — every multiply, every addition.

Each extra hardware unit costs more chip space and more money.

So the question is simple:

“How few hardware units can we get away with?”



Fewer units means:

- Smaller chip
- Lower cost
- Less power use

The Problem, in Plain Words



Think of every chip operation as a small task:

Earliest start

when it's allowed to begin

Deadline

when it must be finished

Duration

how many clock cycles it takes

Order

which tasks must finish first

Goal: fit every task onto as few hardware units as possible, without breaking any deadline or order.

Why We Can't Just Solve It Perfectly

Too Many Possibilities

Trying every possible schedule takes too long once we have more than a handful of operations.

This is a well-known type of problem in computer science — there's no shortcut to checking every option.

With 1000 operations...

0

solutions found by the exact method — even after 10 minutes of computing

We need a smarter, faster approach — Tabu Search.

Meet TS-PRED — Three Simple Steps

1

Prepare

Tighten the schedule using task order, so we know each task's real time window

2

First Try

Place every task one by one, using as few hardware units as possible

3

Improve

Keep tweaking the schedule, trying to use even fewer units

How the “Improve” Step Works

Like rearranging seats to fit everyone at fewer tables



Shake things up

Remove a few tasks from the schedule on purpose



Try a new spot

Slot each removed task back in wherever it fits best



Swap if needed

Trade places with a shorter task if there's no room



Keep what works

Only accept the change if it doesn't need more units

A short “no repeat” rule stops the search from undoing its own progress.

We Built and Tested It in C++

```
struct Job { earliest start, deadline, duration, requirements };  
struct Processor { the tasks assigned to it };  
struct Solution { all processors in use };  
  
// the "no repeat" window shrinks as more units are freed up
```



We didn't just write it — we compiled and ran it.

The next slide shows the real output.

Example: Scheduling 4 Operations

2 multiplications, 2 additions — with a fixed order between them

Task	Type	Starts after
J0	Multiply	—
J1	Multiply	—
J2	Add	J0 and J1
J3	Add	J2

Result: 1 hardware unit is enough — the tasks naturally fit one after another.

```

==== TS-PRED Application Example ====
4 HLS operations, precedence: J0->J2, J1->J2, J2->J3

Initial solution: 1 processors
TS-PRED finished after 7500 iterations.
Best solution: 1 processors

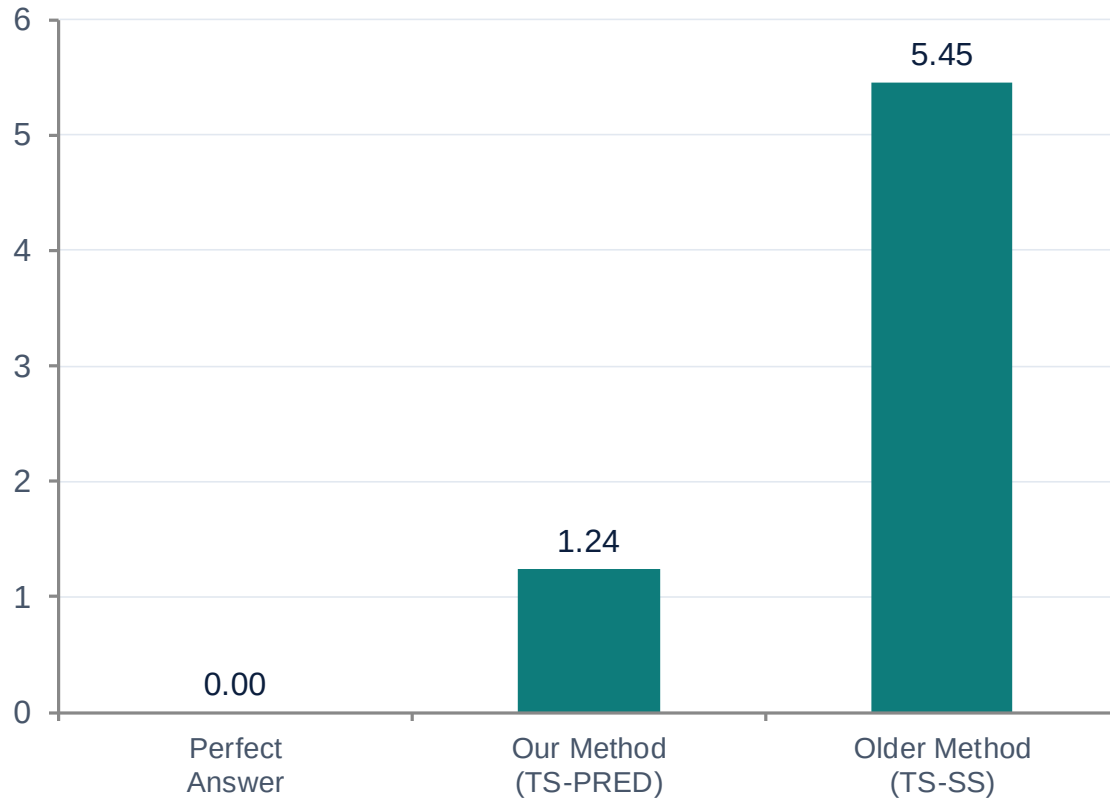
==== Schedule (1 processors) ====
Processor 0: [J0 s=0 f=2] [J1 s=2 f=4] [J2 s=4 f=6] [J3 s=6 f=8]

...Program finished with exit code 0
Press ENTER to exit console.

```

Real, verified output from our C++ program

Does It Work Well?



In short:

- Our method lands within about 1% of the perfect answer
- It's several times faster than the older method
- It even works on problems the exact method can't solve at all

What's Good, What's Not



Strengths

- Always finds a working schedule
- Very close to the perfect answer
- Handles task ordering, unlike older methods



Limitations

- No guarantee it's the perfect answer
- Assumes all hardware units are identical
- Only optimises unit count, not power or area



Future idea: combine with smarter, evolving search methods and support different unit types



Thank You

Questions & Discussion

Tabu Search lets us schedule chip operations using fewer hardware units — fast enough for real designs, and close enough to perfect to trust.